35. Edge Case Hunting & Chaos Testing: Finding Unthinkable Scenarios with AI

Breaking Systems with Leap Years, Negative Balances, and Malicious Unicode
Module 6 • Difficulty Level: Intermediate / Advanced • Estimated Reading Time: 12 min

---

> 🎙️ **NotebookLM & Audio Overview Podcast Briefing**
>
> **Podcast Discussion Hook:**
> *"Why do elite software teams still get blindsided by leap year calculations and integer overflows, and how can AI act as an adversarial chaos engineer?"*
>
> **3 Core Audio Takeaways:**
> - 📌 Real users will never follow your idealized happy path; adversarial and malformed inputs are reality. - 📌 Edge cases encompass: empty strings, boundary numbers (`Number.MAX_SAFE_INTEGER`), timezone shifts, and Unicode payloads. - 📌 Prompting an AI model to generate 10 adversarial edge cases is the highest-ROI defensive programming habit.

📖 1. The Story & Real-World Analogy

Consider an automated banking ATM.

The developers verify standard operations: - Insert card, enter PIN, withdraw $100 -> Success! - Withdraw $500 -> Success!

Now observe an adversarial edge-case scenario: - What happens if someone requests a withdrawal of -$100? If unvalidated, subtracting a negative number increments their account balance! - What if someone injects a 4-byte zero-width Unicode string into the PIN keypad? - What if power drops out the exact millisecond cash leaves the dispenser?

These are **Edge Cases**. Novice developers test only sunny afternoons. Master engineers subject structures to Category 5 hurricanes.

🧠 2. The AI Mental Model & Paradigm Shift

The 5 Primary Edge-Case Taxonomies:

```
1. NUMERIC BOUNDARIES:
   - Zero (0), negative values, floating precision (0.1 + 0.2), Integer overflow limits.

2. STRING & UNICODE PAYLOADS:
   - Empty strings (""), whitespace only ("   "), null/undefined, multi-byte emojis (👨‍👩‍👧‍👦), raw script tags.

3. TEMPORAL ANOMALIES:
   - Leap years (Feb 29), Daylight Saving Time hour shifts, leap seconds, divergent UTC offsets.

4. COLLECTION VOLUMES:
   - Empty lists (`[]`), singletons (`[x]`), high-cardinality collections (1,000,000 items).

5. CONCURRENCY & RACE CONDITIONS:
   - Idempotency violations: double-clicking 'Checkout' within 10 milliseconds.
```

🤖 3. AI Prompt Lab (Production-Ready Templates)

You can directly paste and adapt this prompt template in Cursor, Claude Code, ChatGPT, or Copilot:

````
# ROLE: Chaos & Edge-Case Vulnerability Analyst
Examine this voter eligibility verification method:

## SNIPPET:
```typescript
function isEligibleToVote(birthDateStr: string): boolean {
  const birthYear = new Date(birthDateStr).getFullYear();
  const currentYear = new Date().getFullYear();
  return (currentYear - birthYear) >= 18;
}
````

TASK:

1. Uncover 4 critical edge cases (month/day boundary off-by-one errors, invalid dates, future birthdates).
2. Author adversarial Jest assertions targeting these exact edge cases.
3. Deliver a hardened, bulletproof refactor handling leap years and temporal boundaries.
````

````
---

## 💻 4. Code Walkthrough: Naive / Hallucination vs. Production-Grade

#### ❌ Naive / Unchecked AI Output (Risky):
```typescript
// Kötü Kod: Sadece yılı çıkarır; ay ve günü hesaba katmaz!
// Örnek: Bugün Ocak 2026 ise ve kullanıcı Aralık 2008 doğumluysa (henüz 17 yaşındayken) 18 kabul eder!
export function isEligibleToVoteNaive(birthDateStr: string): boolean {
  return (new Date().getFullYear() - new Date(birthDateStr).getFullYear()) >= 18;
}
````

✅ Senior-Refined Version (Robust & Clean):

```typescript
export function isEligibleToVoteRobust(
  birthDateStr: string,
  now: Date = new Date()
): boolean {
  const birthDate = new Date(birthDateStr);

  // 1. Uç Durum: Geçersiz tarih formatı (NaN)
  if (isNaN(birthDate.getTime())) {
    throw new TypeError("Geçersiz tarih formatı girildi.");
  }

  // 2. Uç Durum: Gelecekte bir doğum tarihi girilemez
  if (birthDate > now) {
    throw new RangeError("Doğum tarihi bugünden ileri bir tarih olamaz.");
  }

  // 3. Kesin Yaş Hesabı (Ay ve Gün Hassasiyeti)
  let age = now.getFullYear() - birthDate.getFullYear();
  const monthDiff = now.getMonth() - birthDate.getMonth();

  if (monthDiff < 0 || (monthDiff === 0 && now.getDate() < birthDate.getDate())) {
    age--; // Doğum günü bu yıl henüz gelmedi!
  }

  // 4. Uç Durum: 130 yaşından büyük mantıksız değerler
  if (age > 130) {
    throw new RangeError("Doğrulanamaz yaş değeri.");
  }

  return age >= 18;
}
```

💡 **Code Analysis:** The hardened version validates unparseable date strings (`NaN`), future timestamps, exact month/day boundaries, and upper-bound sanity checks.
```

⚠️ 5. Junior Pitfalls & Senior Perspective

- **Trap:** Assuming external consumers will always input well-formed data.
  - **Remedy:** Treat incoming parameters with suspicion: validate empty payloads, type mismatches, and boundary sizes.
- **Trap:** Comparing dates via simplistic calendar year subtractions.
  - **Remedy:** Account for exact days and months, or rely on established temporal engines like `date-fns`.
- **Trap:** Neglecting division-by-zero checks in calculation algorithms.
  - **Remedy:** Always assert that denominators are non-zero before executing division or modulo arithmetic.

🎬 6. Udemy Video Lecture Blueprint & Slide Deck

📽️ Lecture Title: *Lecture 35: Edge Case Hunting - Uncovering Impossible Bugs with AI*

- 🖥️ **Slide 1:** Why high-profile outages trace back to neglected boundary edge cases.
- 🖥️ **Slide 2:** The 5 core edge case archetypes (Numeric, Text, Temporal, Collections, Concurrency).
- 🖥️ **Slide 3:** Casting AI as an adversarial chaos engineer to expose software cracks.
- 🖥️ **Slide 4:** Live demo: Breaking and repairing an age-calculation function.

Live Coding / Screencast Scenario:
Submit a string truncate function to an LLM. Prompt: 'Provide 5 adversarial edge-case assertions designed to crash this function'. Watch it expose crashes on empty strings and multi-byte emojis.

🎯 7. Daily Action Task & Mini Quiz

🛠️ Daily Action Task:

Select a critical core function in your project. Prompt an AI model to generate 5 boundary edge cases, and run them against your code.

🧠 Chapter Review Quiz:

Question 1: Why does evaluating `0.1 + 0.2 === 0.3` yield `false` in standard programming runtimes? - A) JavaScript contains a deliberate compiler bug - B) Binary floating-point representation (IEEE 754) introduces fractional precision discrepancies (`0.30000000000000004`) - C) It only manifests on Chrome browsers - D) It actually evaluates to true > **Correct Answer:** B > **Explanation:** Binary arithmetic cannot precisely store fractional base-10 values like 0.1, resulting in rounding discrepancies.

Question 2: What operational failure occurs when division by zero is unguarded in software systems? - A) Processing speeds double - B) The runtime emits a fatal `ZeroDivisionError` or injects corrupting `Infinity` / `NaN` payloads into downstream storage - C) Display monitors disconnect - D) The dividend automatically normalizes to 1 > **Correct Answer:** B > **Explanation:** Division by zero is mathematically undefined, triggering runtime panics or corrupting data pipelines with `NaN` / `Infinity`.